SANPO USB 转 FDCAN

English

SANPO USB 转 FDCAN 协议用于在 Windows 和 Linux 上控制 FDCAN 设备，FDCAN 数据转发给设备，无需驱动。发送报文在 原始FDCAN 数据帧 外附加 SANPO 自定义帧头/尾，附加帧只作为开发板识别使用，下发给 FDCAN 设备的只有原始 FDCAN 数据，不会加入 SANPO 帧头/尾。

SANPO USB 转 经典CAN 请阅读 SANPO USB 转 CAN。

连接

接入开发板USB后，会出现 6 个 USB 串口，其中 2 个管理口用于本协议(Windows 下通常显示为 `SANPO Studio Management Port`)：一个访问 CAN-1/2，另一个访问 CAN-3/4；其余 4 个是 RS485 独立串口。操作系统分配的 `COM` 或 `/dev/ttyACM*` 编号不保证与板上丝印顺序一致，首次使用时建议只连接一个 CAN 设备确认对应关系。
Linux 命令模式下，也可以按照串口识别工具使用说明运行 Python 程序，识别哪 2 个串口是管理口。

示例程序：发送数据前设置 CAN FD 波特率

安装依赖：

```
python -m pip install pyserial
```

下面的示例代码会自动找到并通过2个管理口设置 CAN-1～CAN-4 的波特率。示例把仲裁段设置为 1 Mbps、数据段设置为 5 Mbps。

```
import re
import time
import serial
from serial.tools import list_ports

COMMANDS = ["AT+SETFDCAN=1000000,5000000"]

def is_management_port(port):
    if (port.vid, port.pid) != (0x1209, 0x2323):
        return False
    info = " ".join(str(getattr(port, name, "") or "")
                    for name in ("hwid", "interface", "location")).upper()
    return ("MI_01" in info or "MANAGEMENT PORT" in info or
            re.search(r":(?:X|\d+)\.1(?:\D|$)", info) is not None)

def send_at(stream, command):
    stream.reset_input_buffer()
    stream.write(command.encode("ascii") + b"\r\n")
    stream.flush()
    deadline, reply = time.monotonic() + 1.0, bytearray()
    while time.monotonic() < deadline:
        reply.extend(stream.read(stream.in_waiting or 1))
        if b"OK\r\n" in reply or b"ERR," in reply:
            break
    text = reply.decode("ascii", errors="replace").strip()
    if "OK" not in text:
        raise RuntimeError(f"{stream.port}: {command}: {text or 'no reply'}")
    print(f"{stream.port}: {command} -> {text}")

ports = sorted(p.device for p in list_ports.comports()
               if is_management_port(p))
if len(ports) != 2:
    raise SystemExit(f"Expected 2 SANPO management ports, found {len(ports)}:
{ports}")

for device in ports:
    with serial.Serial(device, 1_000_000, timeout=0.05, write_timeout=1) as stream:
        time.sleep(0.05)
        for command in COMMANDS:
            send_at(stream, command)

print("CAN/FDCAN-1..CAN/FDCAN-4 configured successfully.")
```

需要其他波特率时，修改 `COMMANDS` 中的仲裁段和数据段数字。代码中打开管理口 使用的 `1_000_000` 只是 STM32 CDC 串口参数，不会设置 CAN 或 RS485 的物理波特率； 物理总线参数只由明确的 `AT+SETFDCAN` 命令修改。

参数	支持值
仲裁段	`1000000`、`500000`、`250000`、`125000`、`100000`
数据段	`2000000`、`3000000`、`4000000`、`5000000`

数据长度编码

CAN FD 的数据帧长度字段使用以下编码，支持64字节长度的数据帧：

编码	数据字节数
`00`	0
`01`	1
`02`	2
`03`	3
`04`	4
`05`	5
`06`	6
`07`	7
`08`	8
`09`	12
`0A`	16
`0B`	20
`0C`	24
`0D`	32
`0E`	48
`0F`	64

CAN FD 扩展帧

发送和接收使用相同格式：

字段	长度	说明
帧头	2 字节	固定 `45 46`，即 `EF`
Channel	1 字节	CAN FD 接口编号
CAN ID	4 字节	29 位扩展帧 ID，高字节在前
长度编码	1 字节	`00` ～ `0F`
数据	0～64 字节	字节数由长度编码决定
帧尾	2 字节	固定 `0D 0A`

向 Channel 1、扩展帧 ID `0x1F100101` 发送 16 字节数据：

```
45 46 01 1F 10 01 01 0A
01 01 01 01 42 00 00 00 01 01 64 00 78 56 34 12
0D 0A
```

其中 `0A` 表示数据区为 16 字节。

CAN FD 标准帧

发送和接收使用相同格式：

字段	长度	说明
帧头	2 字节	固定 `53 46`，即 `SF`
Channel	1 字节	CAN FD 接口编号
保留字段	2 字节	固定 `00 00`
CAN ID	2 字节	11 位标准帧 ID，高字节在前
长度编码	1 字节	`00` ～ `0F`
数据	0～64 字节	字节数由长度编码决定
帧尾	2 字节	固定 `0D 0A`

向 Channel 2、标准帧 ID `0x142` 发送 12 字节数据：

```
53 46 02 00 00 01 42 09
01 02 03 04 05 06 07 08 09 0A 0B 0C
0D 0A
```

回包识别

电机反馈会使用同一种 `SF` 或 `EF` 格式返回。根据帧头判断标准帧或扩展帧，根据长度编码确定数据字节数，再使用 Channel 判断来自当前 MCU 的第一路还是第二路。

USB 是字节流，一次串口读取可能包含半帧、完整一帧或多帧。上位机应按帧头和长度编码拆包，不要假设一次 `read()` 恰好得到一帧。

SANPO J8108-R8 FDCAN 示例

完整示例：管理口协议模式 J8108-R8 FDCAN 只读测试

安装 `pyserial` 后运行。程序会自动识别两个 SANPO 管理口，并遍历每个 MCU 的 本地 Channel 1/2，共测试四路 CAN/FDCAN：

```
python3 usb2fdcan_j8108_demo_v8.py
```

程序会向两个管理口发送 `AT+SETFDCAN=1000000,5000000`，再依次通过四条本地 Channel 路径查询电机。它只发送 SANPO FDCAN v1 的 `HELLO` 和 `STATE_REQUEST`，不会使能电机或发送运动目标。 V82 在 FDCAN 参数未变化时只恢复 live FDCAN，不会重复写 Flash。

仅枚举管理口、不设置波特率且不发送测试帧时使用 `--list-only`。

J8108-R8 必须已经选择并重启进入 `SANPO FDCAN Profile`；样例不会切换或写入 电机 Profile。

若系统没有提供完整 USB 端口元数据，或需要缩小排障范围，可显式指定管理口：

```
python3 usb2fdcan_j8108_demo_v8.py \
  --ports /dev/ttyACM0 \
  --channels 1 \
  --node-id 1
```

已有 J8108-R8 如果使用 1M/2M，应增加 `--data-bitrate 2000000`。未接电机的 端口/Channel 显示 `MISS` 属于正常现象。详细说明见 J8108-R8 FDCAN 测试指南。

常见问题

现象	处理方法
CAN FD 没有响应	检查仲裁段、数据段、BRS、终端电阻和对端是否支持 CAN FD
回包 Channel 只显示 1/2	回包使用当前 MCU 的本地编号，属于正常行为
多帧数据连在一起	按帧头、长度编码和帧尾连续解析，不要按串口读取次数分帧